

1. Problem:

Eternity 2 (E2) is an edge-matching puzzle known for being extremely difficult to solve. In fact, the best solution according to online sources so far is 467 matching edges out of 480. Our goal for this project will be to match the most edges on the 16x16 E2 board without having any leftover pieces.

Essentially, the puzzle has 256 unique pieces. Each piece is a simple square with a shape and color for each side (top, bottom, left, right). The shape and color on each side of a piece in the board must match adjacent pieces for the puzzle to be complete. There is a total of 5 edge colours and 17 shapes for a total of 22 possible edges plus the border (23 possibilities).

To render the problem amenable to an algorithmic solution, we will represent each shape and color with a number and assign that number to each edge of our pieces (0 for border, 1-22 for other edges). To match edges, we will only need to match those numbers.

Now that we have a better understanding of the task at hand, we can start thinking about our method to place the pieces on the board. One way is to build the board line by line starting with the leftmost piece. That way, we only need to match the left and top edges of the new piece each time, except for the last piece in the row (right edge must be 0) and for the bottom row (where bottom edge needs to be 0 since we reached the border). For that method, we will create a board with **Border Pieces (their id is -1 to distinguish them and all edges are 0)** around the board. This will cost some memory (a 16x16 board turns to an 18x18 board, more pieces to store) but ultimately will help us when selecting a new individual.

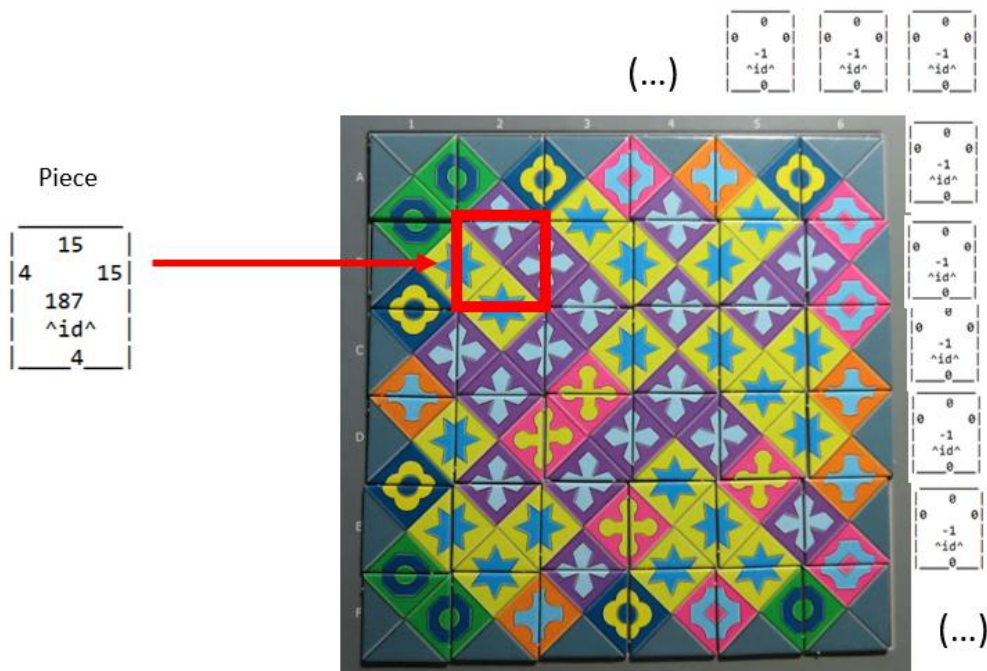


Figure 1: Example of 6x6 E2

1. Modeling:

Special pieces:

Border:

	0	
0		0
	-1	
	^{id}	
	0	

Any piece:

	0	
0		0
	-2	
	^{id}	
	0	

a) Data structures

Piece Class:

First, we need to create a Piece class. This class describes one piece of the puzzle. It is important to note that pieces can be rotated 4 ways, for that we will keep an integer for rotation (0-3). With each rotation, this value is updated (+1) every time we rotate the piece 90 degrees to the right and edges are changed accordingly. Assigning unique ids to each of our pieces will help when verifying the puzzle for matching pieces.

Piecequeue Class:

A queue class for pieces which will contain all our pieces which are not yet placed on the board. Queue is a convenient data structure for this problem since first-in-first-out allows us to iterate through all our pieces by dequeuing and put back unfitting pieces into our queue using the enqueue function (these operations are much faster than using an array of pieces for example, where we would have to either mark that piece or move all pieces in the array when deleting). We can also check if we still have unplaced pieces with the isEmpty() function. We can also be sure that a piece is not used twice in the board. When encountering a placement which is not correct and we need to backtrack, placing the elements back in the queue (enqueue) will make it so those elements have the lowest priority when checking (dequeue) for other pieces to build the line. This will allow the program to remake the line using hopefully new pieces and find a better solution.

Board:

To represent our board, we will use a 2D array of pieces. We could use a simple array, but the 2D array allows us to treat lines and columns separately. All in all, a 2D array for the board makes our algorithm more intuitive. Essentially, the board will be a 2D array (with border pieces).

Puzzle Class:

The puzzle class is our main class and implements all the necessary functions to build and verify the puzzle. It contains our Piecequeue (which contains our pieces), the board (2D array of pieces + border), a random number (used to shuffle our pieces using Knuth shuffle) and 3 integers, numpieces, dimension and backtrack_counter, which respectively represent the total number of pieces, dimensions of the board (with border) and the number of times a line was backtracked for total execution.

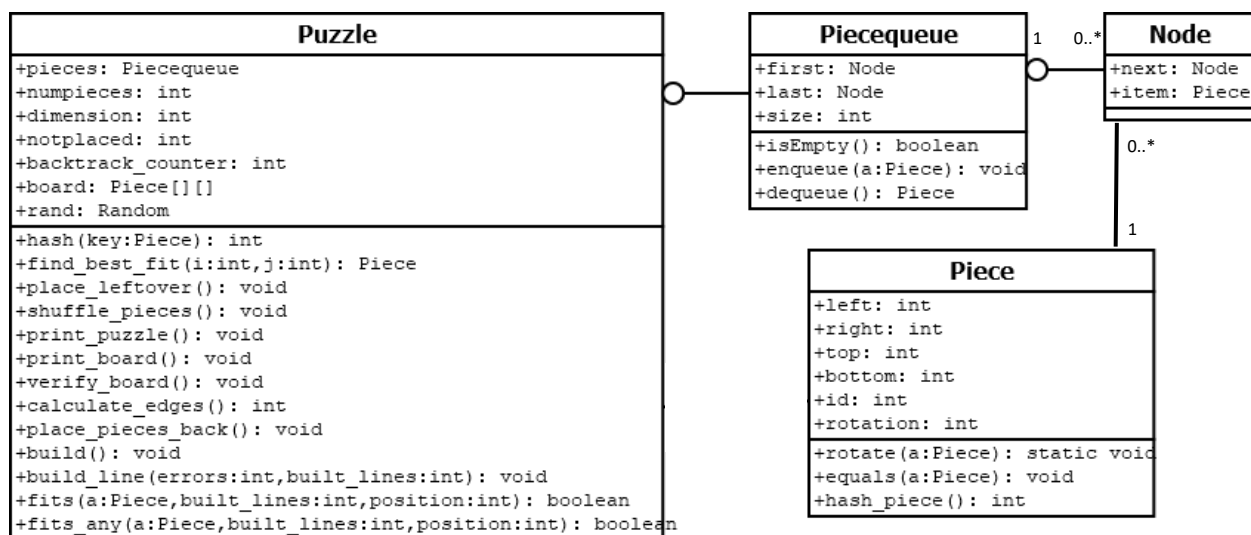


Figure 2: UML

b) Algorithms

- public void build()
*//O(N^2) main function, builds puzzles until we have at least 440/544 matching edges and prints puzzle. (Flow chart in **Fig3**.)*
- public void build_line(int errors, int built_lines)
//O(N^2) builds each line in //puzzle recursively line by line. If at one point there are no pieces that fit, backtrack line and try to make it with different pieces. If after 500 tries there is still no solution, place an any piece (id = -2 this piece

*matches with all edges, essentially the same as leaving the spot empty). (Flow chart in **Fig4.**)*

- public boolean fits(Piece a, int built_lines, int position)
//O(N) Checks if a specific piece fits in the board for a specific position (board[built_lines] [position]), this function checks all rotations for the piece.
- public boolean fits_any(Piece a, int built_lines, int position)
//O(N) Checks if a specific piece fits in the board for a specific position which has any pieces around it (board[built_lines] [position]).
- public void shuffle_pieces()
//O(N) shuffles pieces in Piecequeue using Knuth shuffle this is very important since the order we check our pieces in the Piecequeue has a great influence on our results
- public Piece find_best_fit(int i, int j)
//O(N) Finds best fit (most matching edges) for a specific position in the board amongst our not placed pieces. Used to place leftover pieces (where we placed any pieces in build_line()).
- public void place_leftover()
*//O(N^2) Places leftover pieces in board where we placed any pieces in build_line(). (flow chart in **Fig.5**)*
- public void print_puzzle()
//O(N) Prints whole puzzle with info.
- public void print_board()
//O(N) Prints board.
- public void verify_board()
*//O(N) Verifies if we have pieces with matching ids in the board (not counting border pieces). By using the **hash table** which maps the id using it as an index in an array, we reduced the complexity of this algorithm from $O(N^2)$ to $O(N)$. Unfortunately, since we only use this function once per execution, this does not really result in much improvement.*

- `public int calculate_edges()`
*//O(N) returns the total number of matching edges in the board, since we must match the border too, we actually have 544 edges (480+16*4).*
- `public void place_pieces_back()`
//O(N) Places all the pieces from the board back into the Piecequeue. Essential to rebuild the board.

Figure 3: build() function flow chart

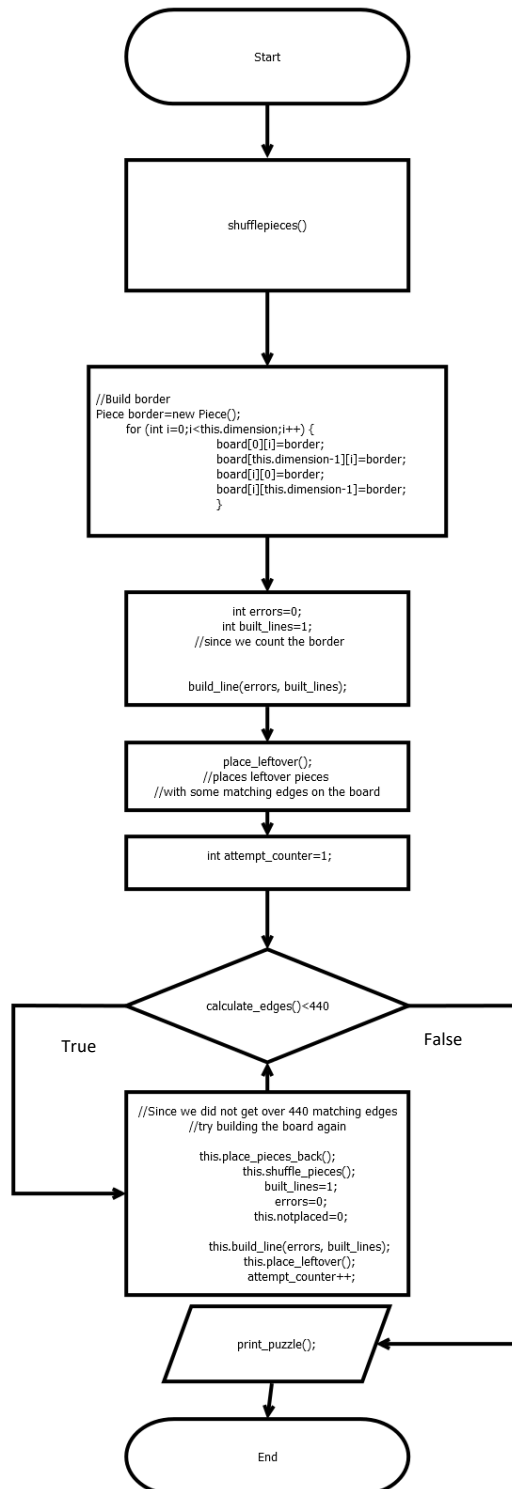


Figure 4: build_line() function flow chart

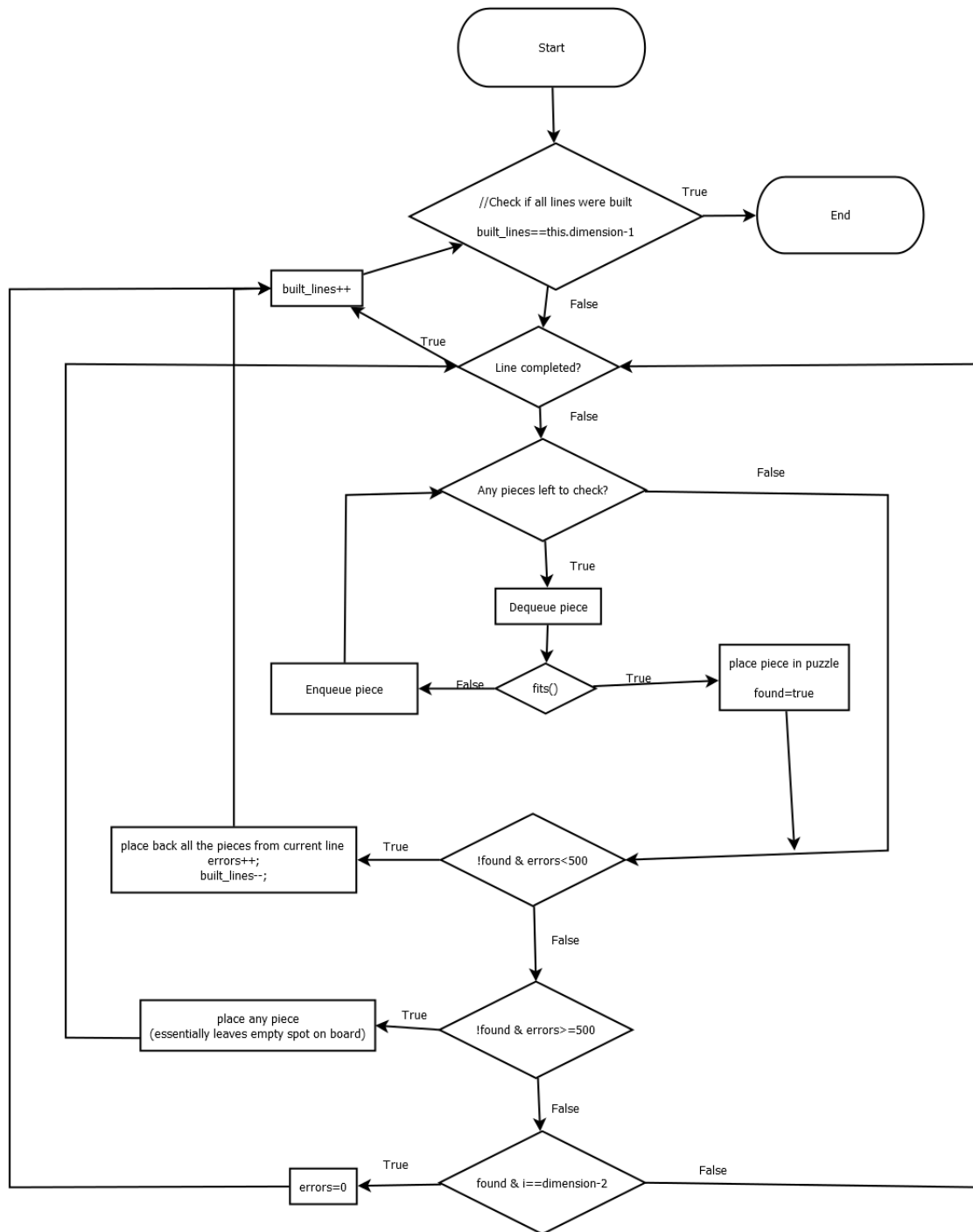
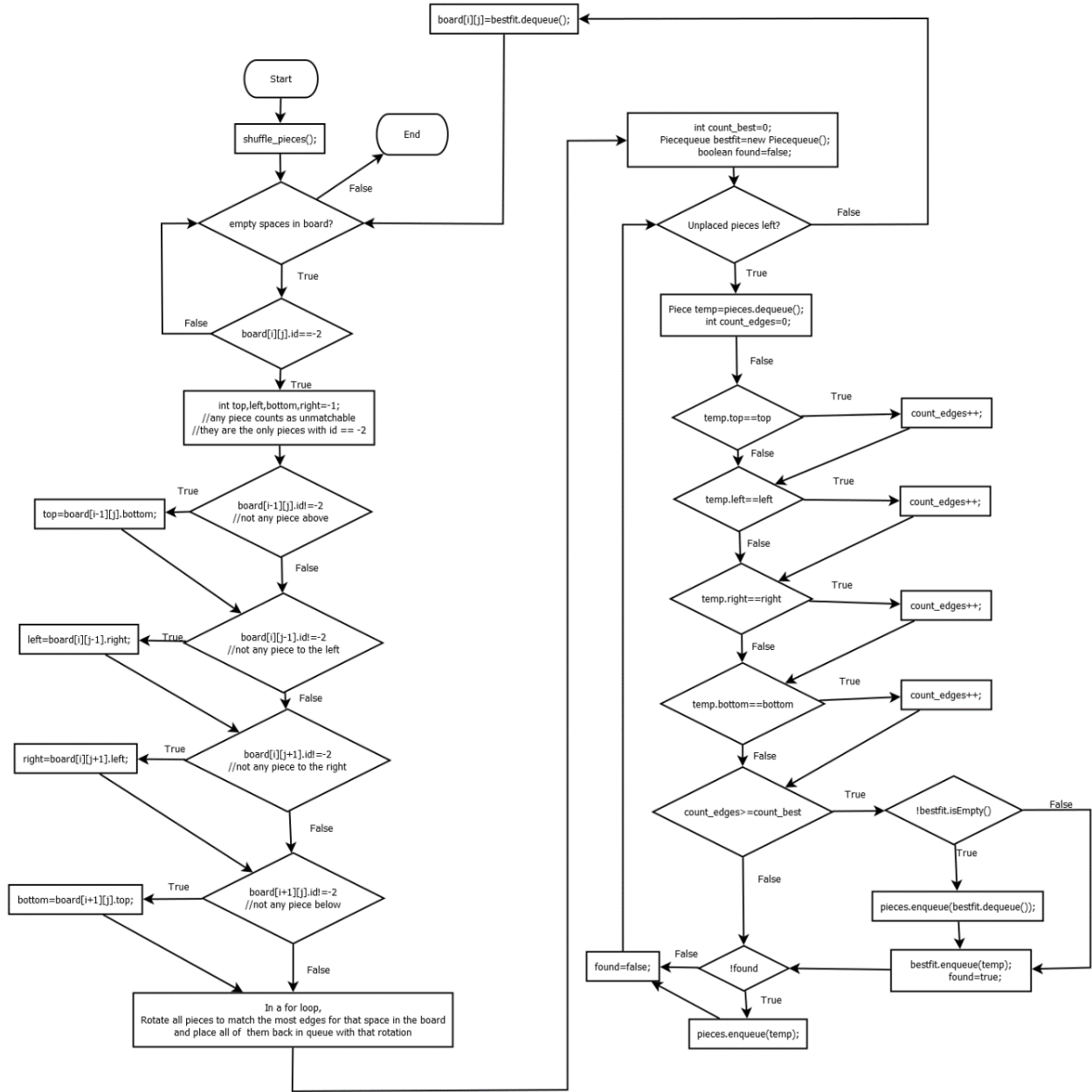


Figure 5: place_leftover() function flow chart



2. Solution:

(Solution for different sized boards in .txt files)

Using the above methods (in build() function), we were able to fit about 440 edges out of 544 in under a minute. Our results vary greatly, we can get anywhere from 350 to 456 matching edges. This is because our result depends on the way we first shuffled our pieces.

Best case:

Technically, if we had all the pieces in the queue in the right order to complete the puzzle, our time complexity would be linear to the number of pieces we have (N). However, that case can be considered impossible to observe, considering the number of pieces we must shuffle.

Average case:

By completing the 16x16 puzzle 10 000 times, we got on average 397 matching edges. The total execution time was 128.860 s which means it took about 13 ms to complete each puzzle.

```
<terminated> Main (1) [Java Application] C:\Program Files\Java\jdk-17\bin\javaw.exe (Dec. 10, 2021, 1:22:43 a.m. - 1:22:49 a.m.)
Matching edges: 393
Matching edges: 409
Matching edges: 405
Matching edges: 404
Matching edges: 409
Matching edges: 397
Matching edges: 395
Matching edges: 403
Matching edges: 403
Matching edges: 386
Matching edges: 386
Matching edges: 391
Matching edges: 397
Matching edges: 401
Matching edges: 391
Matching edges: 389
Matching edges: 410
Matching edges: 395
Matching edges: 410
Matching edges: 397
Matching edges: 383
Matching edges: 439
Matching edges: 411
Matching edges: 415
Matching edges: 380
Matching edges: 408
Matching edges: 404
Matching edges: 368
Matching edges: 414
Matching edges: 451
```

```
-----
TOTAL ATTEMPTS: 463
NUMBER OF TIMES A LINE WAS BACKTRACKED : 1319798 ;
```

For 463 attempts, the number of times a line was backtracked was 1319798, which means on average each line was backtracked 178 times. The average case will be around $\sim 178N^2$.

Worst case:

In the worst case, we would have to backtrack and rebuild each line of the puzzle 500 times. Which means that the time complexity of our algorithm would be $\sim 500N^2$.

Memory:

Each piece takes $4 \times 6 + 16 = 80$ bytes of memory **to store, for 256 pieces (+68 border pieces) we need 25920 bytes of memory (25.92 kilobytes).**

In the Piecequeue with 256 nodes, for each node: 16 (overhead) + 8 (reference to the item) + 8 (reference to next node) = 32 bytes of memory is required, **we have 256 nodes for a total of 8192 bytes (8.192 kilobytes).**

The **Piecequeue** takes $16 + 8$ (first node reference) + 8 (last node reference) + 4 (size integer) + 4 (padding) = **40 bytes of memory.**

The Puzzle class contains 4 integers (16 bytes) + the reference to the Piecequeue (8 bytes) + 4 bytes (for rand since it is technically an integer) + Piece [][] board which contains $18 \times 18 = 324$ pieces (with border) for total of $(324 \times 80 + 8)$ + padding (4 bytes) == $16 + 8 + 4 + 324 \times 80 + 8 + 4 = 25.96$ kilobytes to store.

Total Memory for 16x16:

$$25\,920 + 8\,192 + 40 + 25\,960 = 60\,112 \text{ bytes} = 60 \text{ kilobytes}$$

Total Memory for NxN pieces:

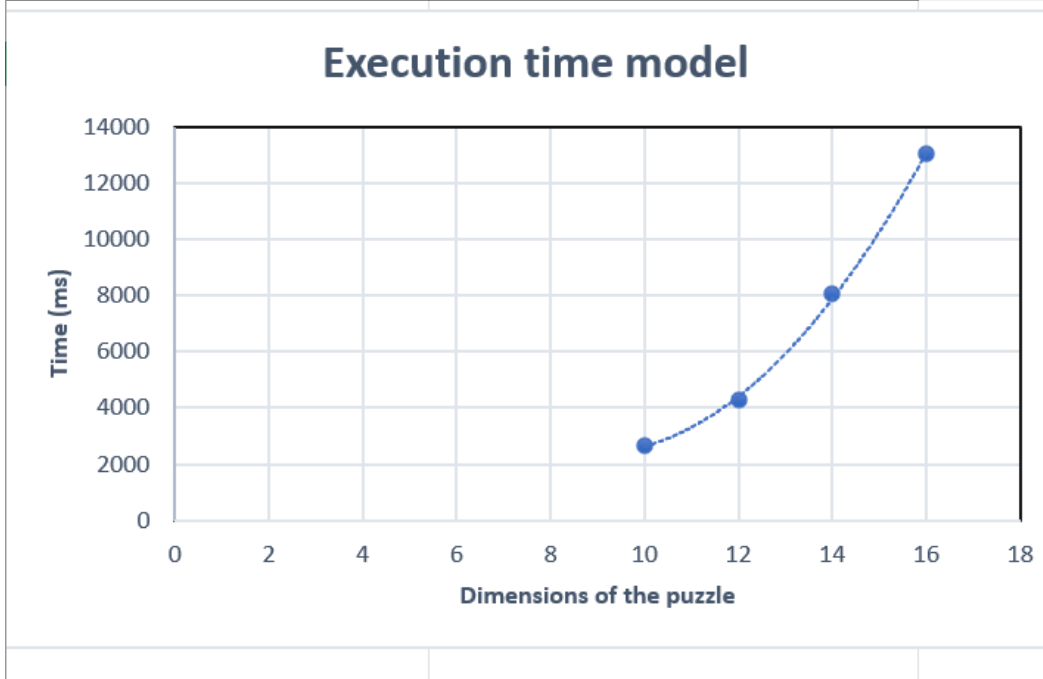
$$80(2+N)^2 + 32N^2 + 40 + 16+8+4+80(2+N)^2+8+4 = \text{total memory for NxN (bytes)} \\ \sim 192N^2$$

We expect a linear model for the memory it takes to solve different sized puzzles.

Time Complexity:

We previously found that the time complexity of our algorithm is $O(N^2)$ (**N being the total number of pieces in the puzzle**), let us plot the time it takes to complete a thousand puzzles for $\text{sqrt}(N)=10, 12, 14, 16$

Dimension of puzzle	time to build it 1000 times (ms)
10	2673
12	4282
14	8047
16	13060



The best fitting trendline model is Polynomial of the 2nd degree, which confirms a quadratic time complexity, $O(N^2)$.

3. Analysis:

The algorithm can place all the pieces in the E2 puzzle and match **80% of edges** under a minute. We verified that the time complexity of the algorithm is **$O(N^2)$** and calculated that the required memory is about **60 kilobytes**.

It is important to note that solving the puzzle using this method is practically impossible. The results depend mostly on the first shuffle. Backtracking is an interesting way to maximize matching edges on the board, but the way I implemented it is not very effective. Only a single line is rebuilt each time, but we most likely would need to change other pieces to advance properly in the puzzle. Another good improvement could have been to start by building the edge of the puzzle, that way we wouldn't need border pieces.

```
<terminated> Test [Java Application] C:\Program Files\Java\jdk-17\bin\javaw.exe (Dec. 6, 2021, 6:05:07 p.m. - 6:06:00 p.m.)
Matching edges: 429
Matching edges: 409
Matching edges: 458
```

_____ ETERNITY II _____

----- BOARD :

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
1	0 3 ^id^ 2	5 40 ^id^ 4	4 8 ^id^ 12	1 30 ^id^ 15	0 6 ^id^ 10	3 48 ^id^ 20	1 32 ^id^ 15	0 15 ^id^ 6	2 7 ^id^ 11	1 58 ^id^ 21	0 11 ^id^ 19	4 46 ^id^ 17	2 25 ^id^ 20	0 31 ^id^ 15	3 21 ^id^ 14	2 0 ^id^ 1
2	0 22 ^id^ 4	17 132 ^id^ 16	19 156 ^id^ 14	8 105 ^id^ 9	7 111 ^id^ 16	20 168 ^id^ 8	12 163 ^id^ 18	5 94 ^id^ 16	8 84 ^id^ 15	19 100 ^id^ 22	12 198 ^id^ 12	10 206 ^id^ 21	22 255 ^id^ 21	20 236 ^id^ 16	13 222 ^id^ 21	11 52 ^id^ 5
3	0 38 ^id^ 4	6 87 ^id^ 18	6 245 ^id^ 20	19 61 ^id^ 7	17 85 ^id^ 9	6 103 ^id^ 17	13 232 ^id^ 20	12 230 ^id^ 13	19 250 ^id^ 18	18 131 ^id^ 7	8 151 ^id^ 13	8 169 ^id^ 18	9 122 ^id^ 16	9 124 ^id^ 20	7 90 ^id^ 17	6 39 ^id^ 4
4	0 41 ^id^ 5	10 69 ^id^ 6	10 239 ^id^ 13	18 120 ^id^ 12	22 181 ^id^ 20	15 175 ^id^ 9	14 187 ^id^ 11	14 226 ^id^ 12	21 121 ^id^ 15	7 102 ^id^ 19	7 223 ^id^ 11	14 110 ^id^ 10	7 123 ^id^ 21	13 240 ^id^ 14	22 173 ^id^ 8	11 42 ^id^ 4
5	0 50 ^id^ 1	9 96 ^id^ 18	9 219 ^id^ 11	16 73 ^id^ 11	6 99 ^id^ 12	20 115 ^id^ 7	12 81 ^id^ 13	22 78 ^id^ 22	11 208 ^id^ 10	6 193 ^id^ 9	22 192 ^id^ 9	18 183 ^id^ 15	21 189 ^id^ 16	11 216 ^id^ 18	13 145 ^id^ 11	8 5 ^id^ 1
6	0 10 ^id^ 3	17 141 ^id^ 22	7 113 ^id^ 20	7 126 ^id^ 7	17 88 ^id^ 6	10 109 ^id^ 13	10 200 ^id^ 15	22 218 ^id^ 17	11 197 ^id^ 15	11 174 ^id^ 13	9 177 ^id^ 10	9 79 ^id^ 12	22 86 ^id^ 22	6 164 ^id^ 20	6 139 ^id^ 21	8 27 ^id^ 3
7	0 35 ^id^ 1	18 190 ^id^ 21	9 176 ^id^ 9	13 108 ^id^ 21	9 65 ^id^ 14	9 179 ^id^ 13	17 159 ^id^ 16	8 162 ^id^ 17	18 60 ^id^ 6	6 80 ^id^ 22	19 211 ^id^ 18	21 144 ^id^ 8	21 63 ^id^ 6	8 207 ^id^ 18	10 228 ^id^ 12	16 45 ^id^ 4
8	0 14 ^id^ 3	22 116 ^id^ 12	7 106 ^id^ 19	8 171 ^id^ 19	15 127 ^id^ 7	15 235 ^id^ 15	16 205 ^id^ 10	18 129 ^id^ 7	15 98 ^id^ 9	20 241 ^id^ 13	22 161 ^id^ 16	8 143 ^id^ 17	19 76 ^id^ 14	12 231 ^id^ 22	17 136 ^id^ 20	7 17 ^id^ 2
9	0 37 ^id^ 4	21 227 ^id^ 14	14 248 ^id^ 16	18 238 ^id^ 13	14 112 ^id^ 12	11 217 ^id^ 21	19 165 ^id^ 8	11 135 ^id^ 15	20 185 ^id^ 11	17 89 ^id^ 6	21 251 ^id^ 19	15 201 ^id^ 14	15 202 ^id^ 14	20 188 ^id^ 9	13 119 ^id^ 7	19 24 ^id^ 2
10	0 44 ^id^ 3	15 247 ^id^ 16	19 244 ^id^ 15	16 150 ^id^ 11	8 152 ^id^ 10	16 220 ^id^ 19	11 142 ^id^ 13	8 158 ^id^ 16	21 133 ^id^ 19	7 83 ^id^ 18	15 249 ^id^ 15	21 170 ^id^ 8	14 224 ^id^ 12	12 182 ^id^ 11	15 118 ^id^ 14	13 53 ^id^ 5
11	0 33 ^id^ 5	15 234 ^id^ 13	20 134 ^id^ 7	9 107 ^id^ 7	11 209 ^id^ 17	12 167 ^id^ 8	10 237 ^id^ 21	16 252 ^id^ 21	8 146 ^id^ 18	15 92 ^id^ 13	16 212 ^id^ 21	19 160 ^id^ 22	10 204 ^id^ 10	8 213 ^id^ 14	11 71 ^id^ 6	17 23 ^id^ 2
12	0 59 ^id^ 2	22 242 ^id^ 14	21 138 ^id^ 11	7 140 ^id^ 17	21 253 ^id^ 18	7 130 ^id^ 16	17 137 ^id^ 16	9 191 ^id^ 18	9 215 ^id^ 16	20 153 ^id^ 8	13 199 ^id^ 10	13 254 ^id^ 22	12 210 ^id^ 10	22 246 ^id^ 21	14 70 ^id^ 21	19 51 ^id^ 5
13	0 6 ^id^ 5	16 82 ^id^ 17	6 222 ^id^ 13	16 128 ^id^ 17	18 91 ^id^ 6	7 221 ^id^ 16	17 184 ^id^ 17	6 167 ^id^ 9	8 104 ^id^ 7	18 114 ^id^ 12	7 178 ^id^ 11	12 26 ^id^ 6	3 125 ^id^ 7	17 66 ^id^ 6	14 97 ^id^ 19	6 20 ^id^ 2
14	0 57 ^id^ 1	16 194 ^id^ 10	10 49 ^id^ 5	7 243 ^id^ 14	15 93 ^id^ 7	22 95 ^id^ 6	10 154 ^id^ 13	10 149 ^id^ 14	13 117 ^id^ 12	9 180 ^id^ 14	21 13 ^id^ 1	4 75 ^id^ 11	0 157 ^id^ 22	6 72 ^id^ 16	14 68 ^id^ 6	10 19 ^id^ 2
15	0 9 ^id^ 4	13 155 ^id^ 8	13 186 ^id^ 9	20 203 ^id^ 10	18 101 ^id^ 10	6 77 ^id^ 18	12 62 ^id^ 15	10 195 ^id^ 10	14 233 ^id^ 16	13 43 ^id^ 0	19 1 ^id^ 0	4 214 ^id^ 20	2 18 ^id^ 3	8 64 ^id^ 9	16 148 ^id^ 19	6 36 ^id^ 3
16	0 2 ^id^ 0	3 28 ^id^ 0	3 4 ^id^ 0	5 166 ^id^ 20	1 56 ^id^ 0	14 47 ^id^ 0	5 54 ^id^ 0	2 74 ^id^ 6	18 34 ^id^ 0	3 147 ^id^ 9	5 172 ^id^ 9	8 55 ^id^ 14	2 225 ^id^ 12	15 196 ^id^ 16	10 29 ^id^ 0	12 1 ^id^ 0